

Databases TD10 - Questions 7-11

Max Hollemans - Arthur Paineau

May 2026

Question 7

If we attempt to transfer funds from an account to itself, the check constraint on the **transfer** table is violated. However, the two **UPDATE** statements on the **account** table will still execute, cancelling each other out and leaving balances unchanged. While this situation does not lead to a crash, it should not be possible to transfer money from an account to itself, therefore we should reject this situation with a clear exception.

If the requested amount exceeds the sender's current balance, the check constraint on the **account** table will be violated. However, the balance check is on the first instruction, so no partial update occurs. Still, it is better to detect this explicitly and raise an explicit exception before executing anything.

Question 8

With the 5-second delay between the two **UPDATE** statements, any client that loads the accounts list between those two updates will see an inconsistent state where the debit has been applied to the sender's account but the credit has not yet reached the recipient. This means that the total amount of money across all accounts appears lower than it actually is for a few seconds.

Question 9

If we kill the Python server with **CTRL+C** at the right time (after the debit update but before the credit update), the server will restart without having completed the transfer, meaning that money has been deducted from the sender's account but never added to the recipient's. This results in a loss of funds in the database. There is no trace of this incomplete operation in the **transfer** history table because the server crashed in the middle of the execution.

Question 10

If two transfers from the same account start at almost the same time, both transactions may read the **transfer** table before either one has written to it. This can lead to issues : for instance, if those two transfers are 1000 € each, both transactions will see that the total transferred amount within the last 24 hours is under the 1000€ limit, and both will proceed. This concurrency issue is a phantom read, where each transaction reads a table that hasn't been updated by the other transaction yet, and it leads to a violation of the 1000€ daily limit rule.

Question 11

Using the **BEGIN TRANSACTION**, **COMMIT** and **ROLLBACK** instructions from PostgreSQL, we can resolve the inconsistencies from exercises 8 and 9. For exercise 8, the transaction ensures that the two **UPDATE** statements are atomic (we either see both changes applied or neither, never an intermediate state). For exercise 9, a server crash in the middle of a transaction automatically triggers a rollback, so the database returns to a consistent state. However, the issue from exercise 10 is not fixed by this alone, as both concurrent transactions can still read the **transfer** table before either commits.